# ASSIGNMENT 01

How to Write a Device Driver

Technical Analysis & Design Report — Task 3 and Task 4

**Student:** Tong Tran Dang

**Student ID:** SE204445

**Target:** ARMv7 / VExpress-A9 QEMU

**Kernel used:** Linux 5.15.0 (existing LAB-02 assets)

**Report status:** Editable review draft — not final submission

Before submission, review the technical content, add personal analysis, update the date if required, and export the final PDF.

# Abstract and scope

This report analyzes two Linux character-device interfaces implemented for Assignment 01. Task 1 extends a character driver with four ioctl commands, read/write access, and a shared kernel/userspace header. Task 2 uses the misc-device framework to expose an atomic counter through `/dev/counter` and publishes statistics through `/proc/counter_info`. The work targets the ARMv7 VExpress-A9 platform under QEMU.

The report focuses on architecture, interface design, synchronization, registration choices, ioctl ABI encoding, test evidence, and the four theory questions. It is not a chronological lab journal. Test results quoted below are transcribed from the saved test outputs and procfs snapshots in the workspace; no screenshots, performance measurements, or missing error logs are invented.

## Scope and reused environment

- Task 1 uses `register_chrdev()` with major 241 and creates a class/device named `asgn1`.
- Task 2 registers a `miscdevice` with dynamic minor allocation, misc major 10, and the name `counter`.
- The previously available kernel image, DTB, and initramfs from LAB-02 were reused by the test scripts. No LAB-01, LAB-02, or LAB-03 source was modified.
- QEMU output and counter snapshots are kept under `logs/`; the final report PDF and submission archive have deliberately not been created before student review.

### What this report demonstrates

The ioctl interface carries command-oriented operations and fixed-size data structures. The misc-device interface is appropriate for a small, single-purpose control/status endpoint. Both expose kernel state through carefully defined read/write behavior, while the implementation uses the kernel's user-copy helpers and explicit cleanup paths.

> **Review notice:** this is an editable draft. The student should verify every statement against the final source and course references, add personal reasoning, and rewrite the material as needed before submission.

# System setup and implementation overview

### Runtime path

Userspace opens device nodes or procfs paths. The system call enters the VFS, which resolves the path and dispatches operations through the registered operation table. The driver validates arguments, accesses kernel-owned state, and uses copy helpers where data crosses the user/kernel boundary.

| Component | Registration and interface | Kernel state / output |
|---|---|---|
| Task 1 `asgn1_driver` | `register_chrdev(241, ...)`; class/device create `/dev/asgn1`; `asgn1_fops` | 1024-byte buffer, mode, five statistics values; operations protected by a mutex |
| Task 2 `counter_driver` | `misc_register()`, `MISC_DYNAMIC_MINOR`; `counter_fops` | `atomic_t` counter plus read/write totals |
| Task 2 proc interface | `proc_create()`; `counter_proc_ops` and `seq_file` | Labelled snapshot at `/proc/counter_info` |

### Test approach represented by the saved evidence

The scripts run in the guest initramfs under QEMU, load the built ARM modules, exercise the requested interfaces, print driver/proc output, unload the module, and include the guest kernel log. Task 1's userspace ioctl test is cross-compiled for ARM. Task 2 uses the guest BusyBox utilities. The scripts assemble a temporary initramfs overlay with only the test-specific files; they do not replace the saved LAB-02 kernel or rootfs.

The saved Task 1 log reports `ASGN1_TEST_RESULT=PASS` and `ASGN1_QEMU_RESULT=PASS`. The saved Task 2 log reports individual PASS checks and `COUNTER_TEST_RESULT=PASS`. The three procfs snapshots were captured from the Task 2 guest run at the required points: before writes, after writing 100, and after reset followed by a read.

## Reproducibility boundary

The report uses only the saved evidence in `logs/test_asgn1_output.txt`, `logs/test_counter_output.txt`, `logs/dmesg_full.txt`, and the three `counter_info_*.txt` files. It does not claim that both modules ran in one QEMU boot: `dmesg_full.txt` labels the two actual post-test captures separately.

# Combined architecture diagram

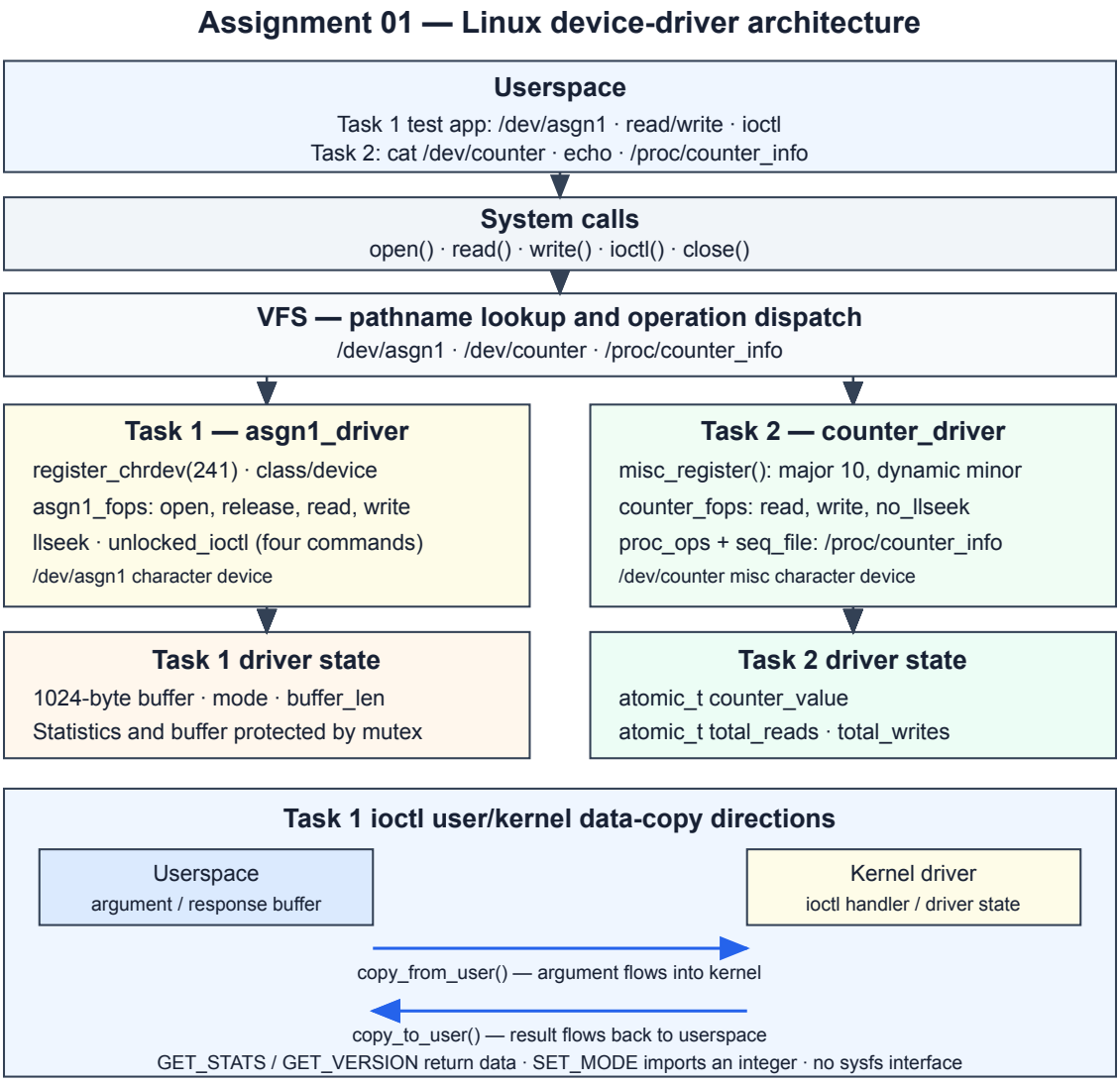



*Figure 1. Combined userspace → system call → VFS → driver architecture, based on the implemented operation tables.*

## How to read the diagram

The Task 1 and Task 2 device paths share the userspace/system-call/VFS layers but dispatch to different operation tables. For `/dev/asgn1`, the VFS uses `asgn1_fops`, including `unlocked_ioctl`. The ioctl

handler chooses one of four commands and accesses the buffer, mode, or statistics. For `/dev/counter`, the VFS dispatches `read` and `write` through `counter_fops`.

The separate `/proc/counter_info` path is handled by procfs operations. Its `seq_file` callback reads the counter and operation totals and formats labelled text. No sysfs attributes are implemented by either assignment driver; sysfs is therefore not shown as an active data path.

### Ioctl data flow

`ioctl(fd, command, argument)` enters the ioctl system call, passes through the VFS, and reaches `asgn1_fops.unlocked_ioctl`. `GET_STATS` and `GET_VERSION` return kernel data with `copy_to_user()`; `SET_MODE` imports the requested integer with `copy_from_user()`. `RESET_BUFFER` has no argument. Unknown command values return `-ENOTTY`.

The vector diagram is provided separately as `bao_cao/architecture.svg` so it can be inspected and edited in a vector graphics tool. Check its labels against the driver sources before exporting the final PDF.

# Task 1 — character-device design

### Registration and lifecycle

The driver uses major 241, within the assignment's selected range, and calls `register_chrdev()` with the name `asgn1`. It then creates a class and device, yielding `/dev/asgn1` when devtmpfs/udev is present. If class or device creation fails, the initialization path releases previously registered resources. On unload, it destroys the device and class and unregisters the character device.

The file-operations table supplies `open`, `release`, `read`, `write`, `llseek`, and `unlocked_ioctl`. The owner field is `THIS_MODULE`. A mutex guards the shared data and counters. Open count is cumulative across successful opens; release logs closure but does not decrement the statistic.

### Buffer behavior

The state includes a fixed 1024-byte buffer and the stored length. Writes replace the current contents. Echo mode (mode 1, default) stores the input bytes as supplied. Kernel mode (mode 0) prepends the literal `[KERNEL]`. A terminating NUL is kept in the internal buffer for safe text-oriented diagnostics, while `buffer_len` represents the number of stored data bytes.

Read checks the file offset and remaining buffer length under the mutex, copies only available data, advances the offset, and increments `read_count` for a successful nonempty read. EOF and failed copies do not increment the counter. The test seeks to the start before reading data back.

### Statistics semantics

`GET_STATS` snapshots `open_count`, `write_count`, `read_count`, `buffer_len`, and `last_write_size` while holding the mutex. It then copies that snapshot to userspace. Reset clears the buffer, write/read counts, and last-write size as implemented; it does not reset cumulative open count or change the mode.

# Task 1 — ioctl ABI and command behavior

### Shared header

`asgn1_ioctl.h` is included by the kernel module and userspace test. It defines magic `'A'`, the version-area constant (64 bytes), and the five-`int` statistics structure. The same macro definitions ensure that both sides agree on command numbers, directions, and encoded argument sizes. The ioctl direction describes data flow from the userspace perspective.

| Command | Macro / encoded type | Implementation and observed test |
|---|---|---|
| `ASGN1_RESET_BUFFER` | `_IO('A', 1)` | Clears data and read/write-related statistics; test confirms empty read and zeroed counts |
| `ASGN1_GET_STATS` | `_IOR('A', 2, struct asgn1_stats)` | Copies a coherent snapshot to userspace; output included five labelled fields |
| `ASGN1_SET_MODE` | `_IOW('A', 3, int)` | Copies an integer from userspace and accepts only 0 or 1 |
| `ASGN1_GET_VERSION` | `_IOR('A', 4, char[64])` | Copies the 64-byte version area containing `asgn1_driver v1.0` |

### Command-number and size fields

The selected magic groups the commands; `_IOC_NR` extracts the sequence number and `_IOC_SIZE` extracts the encoded argument size. For the ARMv7 ABI in this build, the stats structure is 20 bytes (five 4-byte integers), mode is 4 bytes, version is 64 bytes, and reset encodes no argument (size 0). These values describe the ABI; they do not validate a userspace pointer. The driver still uses the uaccess helpers and checks their return values.

### Error handling

The handler returns `-EFAULT` when a user copy fails, `-EINVAL` for a mode outside 0/1, and `-ENOTTY` for an unknown command. The guest test invokes an unknown ioctl and observes userspace `ENOTTY`. Each supported ioctl logs its name for the kernel log.

# Task 2 — misc counter design

## Registration

`counter_driver` is an independent driver, not a derivative of the LAB-02 driver. It initializes a `struct miscdevice` with `MISC_DYNAMIC_MINOR`, name `counter`, and `counter_fops`, then calls `misc_register()`. The framework supplies misc major 10 and a dynamic minor. In the saved QEMU run, the kernel allocated minor 125; that observed value is not hard-coded by the driver.

## Atomic state and read

The counter and cumulative read/write totals are `atomic_t`. A successful read formats the current counter as decimal ASCII followed by newline, copies it to userspace, advances `*ppos` by the returned length, then increments the counter and read total once. If the file offset is nonzero, the callback returns zero to signal EOF. This makes a single `cat /dev/counter` invocation terminate after one logical value.

For a normal serialized reader sequence, the first two `cat` invocations in the test return 0 and 1. Atomic operations prevent lost increments, but the sequence of "read current value, then increment" is not a single atomic transaction across concurrent readers. Simultaneous reads could observe the same value; unique values across concurrent readers are not promised by this simple implementation.

## Write and procfs

Write copies a bounded ASCII input into a kernel buffer, NUL-terminates it, and parses base-10 text with `kstrtoint()`. Negative or malformed values return `-EINVAL`. A valid nonnegative integer sets the counter using `atomic_set()` and increments the write/reset total. A write of zero is the specified reset operation.

`/proc/counter_info` is created with read-only permissions. The `seq_file` callback prints labelled `counter`, `read_count`, and `write_count` values using atomic reads. On module exit, procfs is removed and `misc_deregister()` unregisters the device. If proc creation fails after misc registration, the init path deregisters the misc device before returning an error.

# Device-driver comparison and design choices

| Criterion | Character device | Block device | Network device |
|---|---|---|---|
| Primary interface | `file_operations` dispatched by VFS | `block_device_operations` for device operations plus block/request or blk-mq APIs for I/O | `net_device_ops` and networking subsystem |
| Data unit | Byte stream or driver-defined records/commands | Addressable fixed-size logical blocks/sectors | Packets or frames |
| Buffering | Not inherent; driver or upper layer may buffer | Block layer and page cache commonly queue, merge, or cache I/O | Socket buffers (for example, `sk_buff`) and device/network queues |
| Embedded example | UART, GPIO control interface, assignment counter | eMMC or SD storage presented as sectors | Ethernet MAC or Wi-Fi interface |
| Use when | Byte-oriented I/O or command/status operations | Persistent storage needs sector addressing and block semantics | Data must participate in packet-based network stack |

## Concrete selection examples

1. A UART console transfers a byte stream and control settings, so a character-device interface is appropriate; an eMMC controller exposes logical sectors and should use the block subsystem.
2. An Ethernet MAC exchanges frames and integrates with routing, sockets, and link state, so it should be a network device rather than a character stream.
3. The assignment counter is a small control/status endpoint. A misc character device is sufficient; a block or network interface would add semantics the device does not need.

## `register_chrdev()` compared with `misc_register()`

`register_chrdev()` gives a driver direct control over its character-device major and file operations, useful here because Task 1 requires a selected major number. It also leaves more registration, naming, and cleanup work to the driver. `misc_register()` provides a conventional major 10 and dynamic minor for small miscellaneous devices, reducing boilerplate. It is preferable for a simple unique control device, but not when the driver requires a custom major or a richer subsystem interface.

# Synchronization, ioctl encoding, and Task 1 results

### `atomic_t` versus plain `int`

A plain increment is a read-modify-write sequence. If two CPUs read 7 at the same time, both may compute 8 and store 8; one increment is lost. `atomic_inc()` makes the individual increment indivisible, while `atomic_read()` and `atomic_set()` provide the corresponding atomic accesses required by the task. This protects the counter updates in the tested concurrency model, but it does not make several separate atomic operations one transaction. [2, 5]

## Magic, number, and size

The ioctl magic distinguishes a command family and helps avoid collisions with unrelated commands. `_IOC_NR` is the command ordinal and `_IOC_SIZE` is the encoded argument size. In this header the reset has number 1 and size 0, stats number 2 and size 20, mode number 3 and size 4, and version number 4 and size 64. Use the kernel extraction macros rather than manually decoding the architecture-specific command integer. The magic should be coordinated for a public interface; this assignment uses `'A'` as its local identifier. [2]

## Task 1 measured/observed output

The log records module load with major 241, version `asgn1_driver v1.0`, successful echo and kernel-prefix data checks, all requested ioctl operations, reset behavior, an unsupported command, and unload cleanup. The GET_STATS test output was:

```
open_count=1
write_count=1
read_count=1
buffer_len=9
last_write_size=9
```

Other recorded assertions include `Read verified: [echo-mode]`, `Read verified: [[KERNEL]kernel-mode]`, `PASS: reset clears buffer and counters`, and `PASS: unsupported ioctl returns ENOTTY`. Final result markers: `ASGN1_TEST_RESULT=PASS` and `ASGN1_QEMU_RESULT=PASS`. Source log: `logs/test_asgn1_output.txt`.

# Task 2 test results and procfs snapshots

The Task 2 guest log records misc major 10, dynamic minor 125, a device node at `/dev/counter`, and the module in `/proc/modules`. Reads returned 0 and 1 on successive `cat` operations. Writing 100 set the next value to 100. Writing 0 reset the next value to 0. Negative and nonnumeric input were rejected.

The test verified that unload removed both the device and proc entry. Final marker: `COUNTER_TEST_RESULT=PASS`.

## Three actual snapshots from the QEMU run

### Before writes — `logs/counter_info_initial.txt`

```
counter=0
read_count=0
write_count=0
```

### Immediately after writing 100 — `logs/counter_info_after_write.txt`

```
counter=100
read_count=2
write_count=1
```

### After writing 0 and reading the reset value — `logs/counter_info_final.txt`

```
counter=1
read_count=4
write_count=2
```

The final counter value is 1 because the successful final read returns zero and then increments the stored value. The counts are cumulative since module load: two reads occurred before the after-write snapshot, and the sequence completed with four reads and two valid writes/resets. Rejected writes do not increase the valid write count.

## Interpretation limits

These are functional checks in one ARMv7 QEMU environment, not throughput, latency, or stress-test measurements. The observed dynamic minor 125 applies only to that run. No claim is made about concurrent-reader ordering, real physical hardware, or production workload behavior beyond the implementation and test assertions.

Source log: `logs/test_counter_output.txt`. The three snapshot files are captured test output, not manually invented example values.

# Kernel log evidence and implementation issue

## Kernel messages

`logs/dmesg_full.txt` contains the dmesg output printed after cleanup for each actual QEMU run. The two captures are labelled separately because Task 1 and Task 2 ran in separate guest boots. Task 1 includes messages for module load, each ioctl, data writes, close, and `asgn1_driver: unloaded and cleaned up`. Task 2 includes registration with major 10/minor 125 and `counter_driver: unregistered /dev/counter`.

The complete terminal outputs are preserved in `logs/test_asgn1_output.txt` and `logs/test_counter_output.txt`. The Task 2 log also includes module/device checks, PASS labels, three procfs snapshots, and the dmesg section. The QEMU kernel startup contains unrelated platform-device warnings; they are part of the unmodified system log and were not treated as failures in the assignment tests.

## Issue encountered: seeking for Task 1 read-back test

During the first Task 1 read-back attempt, the test could not seek the device file back to offset zero because the file operations did not provide an `.llseek` handler. The driver was updated to use `default_llseek`; the test then read both echo-mode and kernel-mode data successfully and the final QEMU run passed.

**Evidence limitation:** the original failing run's terminal output was not retained. The successful run replaced the Task 1 log, so there is no preserved original failure message to quote. This report records the issue and fix from the implementation process without fabricating a log excerpt. The saved log proves the successful retest only.

## Debugging lesson

The failure was related to the file-position operation required by the test, not to the ioctl encoding or QEMU environment. Adding the default seek behavior let the test reset the shared file offset and exercise the driver's read path from the start. The successful evidence verifies the behavior that mattered; it does not reproduce or independently prove the discarded initial failure output.

# Theory question 1 — module references and unload

## Question

Kernel modules use reference counting. When does the count increase/decrease? What happens if `rmmod` is used while a process holds the driver open? How can the module be forcibly unloaded? [2, 5]

### Reference answer draft

A module reference is acquired when the kernel successfully takes a reference to code owned by that module and released when that use ends. For character-device file operations, setting `.owner = THIS_MODULE` allows the VFS to hold a module reference while the file is open and release it when the file is closed. Kernel code may also explicitly acquire and release module references where appropriate.

Both assignment operation tables set `.owner = THIS_MODULE`. If a process still has the device open, a normal `rmmod` sees that the module is in use and refuses to unload it, typically reporting "module is in use" or "busy." The exit routine is not run while the reference remains held. The safe procedure is to close all device file descriptors, stop processes using the device, and retry normal unload.

Forced removal with `rmmod -f` is available only if the kernel was configured to permit it. It is unsafe: the kernel may remove code while an open file or callback can still reach it, potentially causing use-after-free, kernel faults, or system instability. It is not an appropriate routine recovery method. Close users first and use normal unload.

### Points to personalize

- Explain the link between the open file and the `.owner` field in the source.
- Distinguish a normal refusal to unload from the conditional, unsafe force option.
- Avoid implying the assignment tests exercised forced removal; they did not.

## Theory question 2 — safe copies across the user/kernel boundary

### Question

Why does `ASGN1_GET_STATS` use `copy_to_user()` rather than `memcpy()` or a direct pointer? What can happen in the MMU if the kernel writes directly to a userspace virtual address?

### Reference answer draft

The ioctl argument is an address supplied by userspace, not a trusted kernel pointer. It may be unmapped, outside the process's valid user region, read-only, or otherwise invalid. A process can also change mappings while a system call is in progress. `memcpy()` performs an ordinary memory copy; it does not provide the kernel's fault-aware user-access handling. Direct dereferencing has the same fundamental problem.

The MMU translates virtual addresses according to the active address space and page permissions. A direct store to an unmapped page or a page without write permission can trigger a page fault or

protection fault in kernel context. An invalid kernel access may produce an oops or compromise system stability instead of returning a clean syscall error. Architecture protections may additionally restrict kernel access to user mappings.

`copy_to_user()` and `copy_from_user()` are the supported user-access helpers. They perform the required fault-aware access and report bytes not copied; the driver checks for a nonzero result and returns `-EFAULT`. In `ASGN1_GET_STATS`, the driver copies a coherent five-field snapshot while holding the mutex, releases the mutex, then calls `copy_to_user()`. The potentially faulting user copy is therefore outside the critical section.

### Points to personalize

Describe the argument as an untrusted userspace virtual address. Do not claim that `copy_to_user()` makes every pointer valid; it allows the driver to handle a failed access safely and report an error.

## Theory question 3 — MTD and block storage

### Question

How does the MTD subsystem differ from a conventional block subsystem? When should embedded storage use MTD, and what is an example of a storage type that requires MTD?

### Reference answer draft

MTD represents raw memory technologies and exposes characteristics of the medium, such as erase-block boundaries, page programming, erase-before-write constraints, and bad blocks. It is commonly used for raw NOR and NAND flash, where software must respect the physical erase/program model and may need wear management and bad-block handling. [2]

A block device presents a logical address space divided into fixed-size sectors/blocks. The block layer queues and schedules block I/O; a translation or flash-management layer may hide the details of the physical medium. Applications and filesystems use block semantics rather than issuing raw erase operations.

Use MTD when the system needs direct access to raw flash semantics—for example, a boot NOR chip or raw NAND managed with UBI/UBIFS. Use a block device for eMMC, SD storage, or flash behind a translation layer when the device is exposed as logical sectors. A storage technology's material alone does not determine the Linux interface; the hardware and management stack determine what is exposed.

### Points to personalize

- Give one concrete raw NAND or NOR example and connect it to erase blocks and bad blocks.

- Contrast it with eMMC or an FTL-backed volume presented as a block device.
- Relate the choice to the interface visible to the driver, not simply “flash versus disk.”

# Theory question 4 — file offset, EOF, and `cat`

## Question

Why must `counter_read()` handle `*ppos`? If it does not check the offset, how does `cat /dev/counter` behave? Explain the repeated VFS read calls and the stopping condition.

## Reference answer draft

`cat` repeatedly calls the `read()` system call and writes each positive-length result to its output. The VFS resolves the open file to the driver's read operation and passes the current file position through `*ppos`. A positive return value means bytes were produced, so `cat` asks for more. A return value of zero means end-of-file and terminates the copy loop; a negative return is an error and also ends it.

If a driver keeps returning a positive value without advancing or checking the position, the next read continues to produce data rather than reaching EOF. For this counter, that could repeatedly emit new decimal values and increment the counter, so `cat` could continue indefinitely until interrupted or an external condition, such as a broken output pipe, stops it.

The implemented callback returns zero if `*ppos` is nonzero. On the first successful read it formats the current counter plus newline, copies the full result, then advances the position by the number of bytes returned. The next VFS-dispatched read sees a nonzero offset and returns zero, which is the EOF condition that lets `cat` exit normally.

## Points to personalize

Connect the explanation directly to the code order: check offset, format, copy, advance offset, increment counter, return byte count. Mention that offset advances only after a successful copy.

# Conclusion, references, and submission review

## Conclusion draft

Task 1 demonstrates command-oriented control through a shared ioctl ABI alongside ordinary read/write operations. Task 2 demonstrates the misc-device framework for a compact control/status device and uses atomic operations for counter state. The QEMU tests confirm the required observable behavior in the saved ARMv7 environment. The proc interface exposes simple labelled statistics, and module cleanup removes the registered interfaces.

These assignment drivers remain demonstrations rather than production designs. A production driver would define concurrency semantics for simultaneous reads/writes, carefully specify buffer limits and partial operations, review access permissions, document ABI compatibility, and add tests for invalid pointers and boundary conditions. Students should add their own analysis of those trade-offs.

## References


1. [1] *ASSIGNMENT01_EmbeddedLinux.docx*, Sections 3.2–3.10 and IV, course-provided assignment specification.
2. [2] Linux Kernel Documentation, Driver API documentation, including character devices, miscellaneous devices, user-space access, procfs/seq_file, atomic types, and MTD. https://docs.kernel.org/driver-api/ and https://docs.kernel.org/core-api/. Consult the version corresponding to the kernel used when finalizing citations.
3. [3] Jonathan Corbet, Alessandro Rubini, and Greg Kroah-Hartman, *Linux Device Drivers*, 3rd ed., O'Reilly, 2005, chapters on character drivers, ioctl, and memory.
4. [4] Linux man-pages project, `ioctl(2)`, `read(2)`, `write(2)`, and `cat(1)` documentation.
5. [5] Robert Love, *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010, sections on concurrency, atomic operations, and modules.


## Section IV file checklist

| Required item | Current status |
|---|---|
| `task1_ioctl/`: C source, ioctl header, Makefile, ARM `.ko`, test script | Present |
| `task2_misc/`: C source, Makefile, ARM `.ko`, test script | Present |
| `logs/test_asgn1_output.txt`, `logs/test_counter_output.txt`, `logs/dmesg_full.txt` | Present; three procfs snapshots also present |
| Final report PDF | Not created; intentionally deferred until student reviews and edits this draft |
| Final `MSSV_HoTen_Asgn01.tar.gz` | Not created; intentionally deferred until report review |

Section IV's directory tree names the in-archive PDF `MSSV_Asgn01_BaoCao.pdf`, while Section 4.4 specifies the standalone submission name `MSSV_HoTen_Asgn01_BaoCao.pdf`. Confirm which internal

name the instructor expects; for the LMS output, follow the explicit Section 4.4 name unless clarified otherwise. The intended archive name for the supplied student details is `SE204445_TongTranDang_Asgn01.tar.gz`.

# Appendix A — Full Task 1 test_asgn1.sh output

```
Running Assignment 01 Task 1 tests in QEMU...
Booting Linux on physical CPU 0x0
Linux version 5.15.0 (elaina@Lenovo-LOQ-15IAX9) (arm-linux-gnueabihf-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0, GNU
ld (GNU Binutils for Ubuntu) 2.46) #1 SMP Sat Sep 12 15:01:16 +07 2026
CPU: ARMv7 Processor [410fc090] revision 0 (ARMv7), cr=10c5387d
CPU: PIPT / VIPT nonaliasing data cache, VIPT nonaliasing instruction cache
OF: fdt: Machine model: V2P-CA9
Memory policy: Data cache writeback
Reserved memory: created DMA memory pool at 0x4c000000, size 8 MiB
OF: reserved mem: initialized node vram@4c000000, compatible id shared-dma-pool
cma: Reserved 16 MiB at 0x6f000000
Zone ranges:
  Normal   [mem 0x0000000060000000-0x000000006fffffff]
Movable zone start for each node
Early memory node ranges
  node   0: [mem 0x0000000060000000-0x000000006fffffff]
Initmem setup node 0 [mem 0x0000000060000000-0x000000006fffffff]
CPU: All CPU(s) started in SVC mode.
percpu: Embedded 15 pages/cpu s30156 r8192 d23092 u61440
Built 1 zonelists, mobility grouping on.  Total pages: 65024
Kernel command line: console=ttyAMA0,115200 rdinit=/init
printk: log_buf_len individual max cpu contribution: 4096 bytes
printk: log_buf_len total cpu_extra contributions: 12288 bytes
printk: log_buf_len min size: 16384 bytes
printk: log_buf_len: 32768 bytes
printk: early log buf free: 14964(91%)
Dentry cache hash table entries: 32768 (order: 5, 131072 bytes, linear)
Inode-cache hash table entries: 16384 (order: 4, 65536 bytes, linear)
mem auto-init: stack:all(zero), heap alloc:off, heap free:off
Memory: 229228K/262144K available (8192K kernel code, 593K rwdata, 1784K rodata, 1024K init, 151K bss, 16532K
reserved, 16384K cma-reserved)
SLUB: HWalign=64, Order=0-3, MinObjects=0, CPUs=4, Nodes=1
trace event string verifier disabled
rcu: Hierarchical RCU implementation.
rcu:            RCU event tracing is enabled.
rcu:            RCU restricting CPUs from NR_CPUS=8 to nr_cpu_ids=4.
rcu: RCU calculated value of scheduler-enlistment delay is 10 jiffies.
rcu: Adjusting geometry for rcu_fanout_leaf=16, nr_cpu_ids=4
NR_IRQS: 16, nr_irqs: 16, preallocated irqs: 16
GIC CPU mask not found - kernel will fail to boot.
GIC CPU mask not found - kernel will fail to boot.
L2C: platform modifies aux control register: 0x02020000 -> 0x02420000
L2C: DT/platform modifies aux control register: 0x02020000 -> 0x02420000
L2C-310 enabling early BRESP for Cortex-A9
L2C-310 full line of zeros enabled for Cortex-A9
L2C-310 dynamic clock gating disabled, standby mode disabled
L2C-310 cache controller enabled, 8 ways, 128 kB
L2C-310: CACHE_ID 0x410000c8, AUX_CTRL 0x46420001
random: get_random_bytes called from start_kernel+0x4f8/0x6ac with crng_init=0
sched_clock: 32 bits at 24MHz, resolution 41ns, wraps every 89478484971ns
clocksource: arm,sp804: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 1911260446275 ns
Failed to initialize '/bus@40000000/motherboard-bus@40000000/iofpga@7,00000000/timer@12000': -22
smp_twd: clock not found -2
Console: colour dummy device 80x30
Calibrating local timer... 94.87MHz.
Calibrating delay loop... 1646.59 BogoMIPS (lpj=8232960)
pid_max: default: 32768 minimum: 301
Mount-cache hash table entries: 1024 (order: 0, 4096 bytes, linear)
Mountpoint-cache hash table entries: 1024 (order: 0, 4096 bytes, linear)
CPU: Testing write buffer coherency: ok
CPU0: Spectre v2: using BPIALL workaround
CPU0: thread -1, cpu 0, socket 0, mpidr 80000000
Setting up static identity map for 0x60100000 - 0x60100060
rcu: Hierarchical SRCU implementation.
smp: Bringing up secondary CPUs ...
smp: Brought up 1 node, 1 CPU
SMP: Total of 1 processors activated (1646.59 BogoMIPS).
CPU: All CPU(s) started in SVC mode.
devtmpfs: initialized
VFP support v0.3: implementor 41 architecture 3 part 30 variant 9 rev 0
clocksource: jiffies: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 19112604462750000 ns
futex hash table entries: 1024 (order: 4, 65536 bytes, linear)
NET: Registered PF_NETLINK/PF_ROUTE protocol family
DMA: preallocated 256 KiB pool for atomic coherent allocations
cpuidle: using governor ladder
hw-breakpoint: debug architecture 0x4 unsupported.
Serial: AMBA PL011 UART driver
irq: type mismatch, failed to map hwirq-75 for interrupt-controller@1e001000!
SCSI subsystem initialized
usbcore: registered new interface driver usbfs
usbcore: registered new interface driver hub
usbcore: registered new device driver usb
pps_core: LinuxPPS API ver. 1 registered
pps_core: Software ver. 5.3.6 - Copyright 2005-2007 Rodolfo Giometti <giometti@linux.it>
PTP clock support registered
Advanced Linux Sound Architecture Driver Initialized.
clocksource: Switched to clocksource arm,sp804
NET: Registered PF_INET protocol family
IP idents hash table entries: 4096 (order: 3, 32768 bytes, linear)
tcp_listen_portaddr_hash hash table entries: 512 (order: 0, 6144 bytes, linear)
TCP established hash table entries: 2048 (order: 1, 8192 bytes, linear)
TCP bind hash table entries: 2048 (order: 2, 16384 bytes, linear)
TCP: Hash tables configured (established 2048 bind 2048)
UDP hash table entries: 256 (order: 1, 8192 bytes, linear)
UDP-Lite hash table entries: 256 (order: 1, 8192 bytes, linear)
NET: Registered PF_UNIX/PF_LOCAL protocol family
RPC: Registered named UNIX socket transport module.
RPC: Registered udp transport module.
RPC: Registered tcp transport module.
RPC: Registered tcp NFSv4.1 backchannel transport module.
Unpacking initramfs...
hw perfevents: enabled with armv7_cortex_a9 PMU driver, 7 counters available
workingset: timestamp_bits=30 max_order=16 bucket_order=0
squashfs: version 4.0 (2009/01/31) Phillip Lougher
jffs2: version 2.2. (NAND) © 2001-2006 Red Hat, Inc.
9p: Installing v9fs 9p2000 file system support
io scheduler mq-deadline registered
io scheduler kyber registered
sii902x 0-0060: supply iovcc not found, using dummy regulator
sii902x 0-0060: supply cvcc12 not found, using dummy regulator
Freeing initrd memory: 1748K
physmap-flash 40000000.flash: physmap platform flash device: [mem 0x40000000-0x43ffffff]
40000000.flash: Found 2 x16 devices at 0x0 in 32-bit bank. Manufacturer ID 0x000000 Chip ID 0x000000
Intel/Sharp Extended Query Table at 0x0031
Using buffer write method
physmap-flash 44000000.flash: physmap platform flash device: [mem 0x44000000-0x47ffffff]
40000000.flash: Found 2 x16 devices at 0x0 in 32-bit bank. Manufacturer ID 0x000000 Chip ID 0x000000
Intel/Sharp Extended Query Table at 0x0031
Using buffer write method
Concatenating MTD devices:
(0): "40000000.flash"
(1): "40000000.flash"
into device "40000000.flash"
physmap-flash 48000000.psram: physmap platform flash device: [mem 0x48000000-0x49ffffff]
libphy: Fixed MDIO Bus: probed
libphy: smsc911x-mdio: probed
smsc911x 4e000000.ethernet eth0: MAC Address: 52:54:00:12:34:56
isp1760 4f000000.usb: isp1760 bus width: 32, oc: digital
isp1760 4f000000.usb: NXP ISP1760 USB Host Controller
isp1760 4f000000.usb: new USB bus registered, assigned bus number 1
isp1760 4f000000.usb: Scratch test failed. 0x00000000
isp1760 4f000000.usb: can't setup: -19
isp1760 4f000000.usb: USB bus 1 deregistered
usbcore: registered new interface driver usb-storage
ledtrig-cpu: registered to indicate activity on CPUs
usbcore: registered new interface driver usbhid
usbhid: USB HID core driver
NET: Registered PF_PACKET protocol family
9pnet: Installing 9P2000 support
Registering SWP/SWPB emulation handler
aaci-pl041 10004000.aaci: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 32
aaci-pl041 10004000.aaci: FIFO 512 entries
mmci-pl18x 10005000.mmci: Got CD GPIO
mmci-pl18x 10005000.mmci: Got WP GPIO
mmci-pl18x 10005000.mmci: mmc0: PL181 manf 41 rev0 at 0x10005000 irq 33,34 (pio)
10009000.uart: ttyAMA0 at MMIO 0x10009000 (irq = 37, base_baud = 0) is a PL011 rev1
printk: console [ttyAMA0] enabled
1000a000.uart: ttyAMA1 at MMIO 0x1000a000 (irq = 38, base_baud = 0) is a PL011 rev1
```

```
1000b000.uart: ttyAMA2 at MMIO 0x1000b000 (irq = 39, base_baud = 0) is a PL011 rev1
1000c000.uart: ttyAMA3 at MMIO 0x1000c000 (irq = 40, base_baud = 0) is a PL011 rev1
rtc-pl031 10017000.rtc: registered as rtc0
rtc-pl031 10017000.rtc: setting system clock to 2026-11-06T06:06:49 UTC (1793945209)
amba 10020000.clcd: Fixing up cyclic dependency with 0-0039
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
sii902x 0-0039: supply iovcc not found, using dummy regulator
sii902x 0-0039: supply cvcc12 not found, using dummy regulator
i2c i2c-0: Added multiplexed i2c bus 2
drm-clcd-pl111 1001f000.clcd: assigned reserved memory node vram@4c000000
drm-clcd-pl111 1001f000.clcd: using device-specific reserved memory
drm-clcd-pl111 1001f000.clcd: core tile graphics present
drm-clcd-pl111 1001f000.clcd: this device will be deactivated
drm-clcd-pl111 1001f000.clcd: Versatile Express init failed - -19
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
drm-clcd-pl111 10020000.clcd: found bridge on endpoint 0
drm-clcd-pl111 10020000.clcd: Using non-panel bridge
[drm] Initialized pl111 1.0.0 20170317 for 10020000.clcd on minor 0
input: AT Raw Set 2 keyboard as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@40000000/
bus@40000000:motherboard-bus@40000000:iofpga@7,00000000/10006000.kmi/serio0/input/input0
Console: switching to colour frame buffer device 128x48
drm-clcd-pl111 10020000.clcd: [drm] fb0: pl111 frame buffer device
ALSA device list:
  #0: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 32
Freeing unused kernel image (initmem) memory: 1024K
Run /init as init process
asgn1_driver: loading out-of-tree module taints kernel.
asgn1_driver: loaded with major 241
asgn1_driver: device opened
asgn1_driver: ASGN1_RESET_BUFFER called
asgn1_driver: ASGN1_GET_VERSION called
Version: asgn1_driver v1.0
asgn1_driver: ASGN1_SET_MODE called
asgn1_driver: stored 9 input bytes in mode 1
Read verified: [echo-mode]
asgn1_driver: ASGN1_GET_STATS called
open_count=1
write_count=1
read_count=1
buffer_len=9
last_write_size=9
PASS: GET_STATS and echo mode
asgn1_driver: ASGN1_SET_MODE called
asgn1_driver: stored 11 input bytes in mode 0
Read verified: [[KERNEL]kernel-mode]
PASS: kernel mode prefix
asgn1_driver: ASGN1_RESET_BUFFER called
asgn1_driver: ASGN1_GET_STATS called
PASS: reset clears buffer and counters
asgn1_driver: unsupported ioctl 0x4163
PASS: unsupported ioctl returns ENOTTY
asgn1_driver: device closed
ASGN1_TEST_RESULT=PASS
asgn1_driver: unloaded and cleaned up
----- dmesg after driver cleanup -----
Booting Linux on physical CPU 0x0
Linux version 5.15.0 (elaina@Lenovo-LOQ-15IAX9) (arm-linux-gnueabihf-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0, GNU
ld (GNU Binutils for Ubuntu) 2.46) #1 SMP Sat Sep 12 15:01:16 +07 2026
CPU: ARMv7 Processor [410fc090] revision 0 (ARMv7), cr=10c5387d
CPU: PIPT / VIPT nonaliasing data cache, VIPT nonaliasing instruction cache
OF: fdt: Machine model: V2P-CA9
Memory policy: Data cache writeback
Reserved memory: created DMA memory pool at 0x4c000000, size 8 MiB
OF: reserved mem: initialized node vram@4c000000, compatible id shared-dma-pool
cma: Reserved 16 MiB at 0x6f000000
Zone ranges:
  Normal   [mem 0x0000000060000000-0x000000006fffffff]
Movable zone start for each node
Early memory node ranges
  node   0: [mem 0x0000000060000000-0x000000006fffffff]
Initmem setup node 0 [mem 0x0000000060000000-0x000000006fffffff]
CPU: All CPU(s) started in SVC mode.
percpu: Embedded 15 pages/cpu s30156 r8192 d23092 u61440
pcpu-alloc: s30156 r8192 d23092 u61440 alloc=15*4096
pcpu-alloc: [0] 0 [0] 1 [0] 2 [0] 3
Built 1 zonelists, mobility grouping on.  Total pages: 65024
Kernel command line: console=ttyAMA0,115200 rdinit=/init
printk: log_buf_len individual max cpu contribution: 4096 bytes
printk: log_buf_len total cpu_extra contributions: 12288 bytes
printk: log_buf_len min size: 16384 bytes
printk: log_buf_len: 32768 bytes
printk: early log buf free: 14964(91%)
Dentry cache hash table entries: 32768 (order: 5, 131072 bytes, linear)
Inode-cache hash table entries: 16384 (order: 4, 65536 bytes, linear)
mem auto-init: stack:all(zero), heap alloc:off, heap free:off
Memory: 229228K/262144K available (8192K kernel code, 593K rwdata, 1784K rodata, 1024K init, 151K bss, 16532K
reserved, 16384K cma-reserved)
SLUB: HWalign=64, Order=0-3, MinObjects=0, CPUs=4, Nodes=1
trace event string verifier disabled
rcu: Hierarchical RCU implementation.
rcu:            RCU event tracing is enabled.
rcu:            RCU restricting CPUs from NR_CPUS=8 to nr_cpu_ids=4.
rcu: RCU calculated value of scheduler-enlistment delay is 10 jiffies.
rcu: Adjusting geometry for rcu_fanout_leaf=16, nr_cpu_ids=4
NR_IRQS: 16, nr_irqs: 16, preallocated irqs: 16
GIC CPU mask not found - kernel will fail to boot.
GIC CPU mask not found - kernel will fail to boot.
L2C: platform modifies aux control register: 0x02020000 -> 0x02420000
L2C: DT/platform modifies aux control register: 0x02020000 -> 0x02420000
L2C-310 enabling early BRESP for Cortex-A9
L2C-310 full line of zeros enabled for Cortex-A9
L2C-310 dynamic clock gating disabled, standby mode disabled
L2C-310 cache controller enabled, 8 ways, 128 kB
L2C-310: CACHE_ID 0x410000c8, AUX_CTRL 0x46420001
random: get_random_bytes called from start_kernel+0x4f8/0x6ac with crng_init=0
sched_clock: 32 bits at 24MHz, resolution 41ns, wraps every 89478484971ns
clocksource: arm,sp804: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 1911260446275 ns
Failed to initialize '/bus@40000000/motherboard-bus@40000000/iofpga@7,00000000/timer@12000': -22
smp_twd: clock not found -2
Console: colour dummy device 80x30
Calibrating local timer... 94.87MHz.
Calibrating delay loop... 1646.59 BogoMIPS (lpj=8232960)
pid_max: default: 32768 minimum: 301
Mount-cache hash table entries: 1024 (order: 0, 4096 bytes, linear)
Mountpoint-cache hash table entries: 1024 (order: 0, 4096 bytes, linear)
CPU: Testing write buffer coherency: ok
CPU0: Spectre v2: using BPIALL workaround
CPU0: thread -1, cpu 0, socket 0, mpidr 80000000
Setting up static identity map for 0x60100000 - 0x60100060
rcu: Hierarchical SRCU implementation.
smp: Bringing up secondary CPUs ...
smp: Brought up 1 node, 1 CPU
SMP: Total of 1 processors activated (1646.59 BogoMIPS).
CPU: All CPU(s) started in SVC mode.
devtmpfs: initialized
VFP support v0.3: implementor 41 architecture 3 part 30 variant 9 rev 0
clocksource: jiffies: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 19112604462750000 ns
futex hash table entries: 1024 (order: 4, 65536 bytes, linear)
NET: Registered PF_NETLINK/PF_ROUTE protocol family
DMA: preallocated 256 KiB pool for atomic coherent allocations
cpuidle: using governor ladder
hw-breakpoint: debug architecture 0x4 unsupported.
Serial: AMBA PL011 UART driver
irq: type mismatch, failed to map hwirq-75 for interrupt-controller@1e001000!
SCSI subsystem initialized
libata version 3.00 loaded.
usbcore: registered new interface driver usbfs
usbcore: registered new interface driver hub
usbcore: registered new device driver usb
pps_core: LinuxPPS API ver. 1 registered
pps_core: Software ver. 5.3.6 - Copyright 2005-2007 Rodolfo Giometti <giometti@linux.it>
PTP clock support registered
Advanced Linux Sound Architecture Driver Initialized.
clocksource: Switched to clocksource arm,sp804
NET: Registered PF_INET protocol family
IP idents hash table entries: 4096 (order: 3, 32768 bytes, linear)
tcp_listen_portaddr_hash hash table entries: 512 (order: 0, 6144 bytes, linear)
TCP established hash table entries: 2048 (order: 1, 8192 bytes, linear)
TCP bind hash table entries: 2048 (order: 2, 16384 bytes, linear)
TCP: Hash tables configured (established 2048 bind 2048)
UDP hash table entries: 256 (order: 1, 8192 bytes, linear)
UDP-Lite hash table entries: 256 (order: 1, 8192 bytes, linear)
NET: Registered PF_UNIX/PF_LOCAL protocol family
RPC: Registered named UNIX socket transport module.
RPC: Registered udp transport module.
RPC: Registered tcp transport module.
RPC: Registered tcp NFSv4.1 backchannel transport module.
Unpacking initramfs...
```

```
hw perfevents: enabled with armv7_cortex_a9 PMU driver, 7 counters available
workingset: timestamp_bits=30 max_order=16 bucket_order=0
squashfs: version 4.0 (2009/01/31) Phillip Lougher
jffs2: version 2.2. (NAND) © 2001-2006 Red Hat, Inc.
9p: Installing v9fs 9p2000 file system support
io scheduler mq-deadline registered
io scheduler kyber registered
sii902x 0-0060: supply iovcc not found, using dummy regulator
sii902x 0-0060: supply cvcc12 not found, using dummy regulator
Freeing initrd memory: 1748K
physmap-flash 40000000.flash: physmap platform flash device: [mem 0x40000000-0x43ffffff]
40000000.flash: Found 2 x16 devices at 0x0 in 32-bit bank. Manufacturer ID 0x000000 Chip ID 0x000000
Intel/Sharp Extended Query Table at 0x0031
Using buffer write method
erase region 0: offset=0x0,size=0x40000,blocks=256
physmap-flash 40000000.flash: physmap platform flash device: [mem 0x44000000-0x47ffffff]
40000000.flash: Found 2 x16 devices at 0x0 in 32-bit bank. Manufacturer ID 0x000000 Chip ID 0x000000
Intel/Sharp Extended Query Table at 0x0031
Using buffer write method
erase region 0: offset=0x0,size=0x40000,blocks=256
Concatenating MTD devices:
(0): "40000000.flash"
(1): "40000000.flash"
into device "40000000.flash"
physmap-flash 48000000.psram: physmap platform flash device: [mem 0x48000000-0x49ffffff]
libphy: Fixed MDIO Bus: probed
libphy: smsc911x-mdio: probed
smsc911x 4e000000.ethernet eth0: MAC Address: 52:54:00:12:34:56
isp1760 4f000000.usb: isp1760 bus width: 32, oc: digital
isp1760 4f000000.usb: NXP ISP1760 USB Host Controller
isp1760 4f000000.usb: new USB bus registered, assigned bus number 1
isp1760 4f000000.usb: Scratch test failed. 0x00000000
isp1760 4f000000.usb: can't setup: -19
isp1760 4f000000.usb: USB bus 1 deregistered
usbcore: registered new interface driver usb-storage
ledtrig-cpu: registered to indicate activity on CPUs
usbcore: registered new interface driver usbhid
usbhid: USB HID core driver
NET: Registered PF_PACKET protocol family
9pnet: Installing 9P2000 support
Registering SWP/SWPB emulation handler
aaci-pl041 10004000.aaci: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 32
aaci-pl041 10004000.aaci: FIFO 512 entries
mmci-pl18x 10005000.mmci: Got CD GPIO
mmci-pl18x 10005000.mmci: Got WP GPIO
mmci-pl18x 10005000.mmci: mmc0: PL181 manf 41 rev0 at 0x10005000 irq 33,34 (pio)
10009000.uart: ttyAMA0 at MMIO 0x10009000 (irq = 37, base_baud = 0) is a PL011 rev1
printk: console [ttyAMA0] enabled
1000a000.uart: ttyAMA1 at MMIO 0x1000a000 (irq = 38, base_baud = 0) is a PL011 rev1
1000b000.uart: ttyAMA2 at MMIO 0x1000b000 (irq = 39, base_baud = 0) is a PL011 rev1
1000c000.uart: ttyAMA3 at MMIO 0x1000c000 (irq = 40, base_baud = 0) is a PL011 rev1
rtc-pl031 10017000.rtc: registered as rtc0
rtc-pl031 10017000.rtc: setting system clock to 2026-11-06T06:06:49 UTC (1793945209)
amba 10020000.clcd: Fixing up cyclic dependency with 0-0039
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
sii902x 0-0039: supply iovcc not found, using dummy regulator
sii902x 0-0039: supply cvcc12 not found, using dummy regulator
i2c i2c-0: Added multiplexed i2c bus 2
drm-clcd-pl111 1001f000.clcd: assigned reserved memory node vram@4c000000
drm-clcd-pl111 1001f000.clcd: using device-specific reserved memory
drm-clcd-pl111 1001f000.clcd: core tile graphics present
drm-clcd-pl111 1001f000.clcd: this device will be deactivated
drm-clcd-pl111 1001f000.clcd: Versatile Express init failed - -19
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
drm-clcd-pl111 10020000.clcd: found bridge on endpoint 0
drm-clcd-pl111 10020000.clcd: Using non-panel bridge
[drm] Initialized pl111 1.0.0 20170317 for 10020000.clcd on minor 0
input: AT Raw Set 2 keyboard as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@40000000/
bus@40000000:motherboard-bus@40000000:iofpga@7,00000000/10006000.kmi/serio0/input/input0
Console: switching to colour frame buffer device 128x48
drm-clcd-pl111 10020000.clcd: [drm] fb0: pl111 frame buffer device
ALSA device list:
  #0: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 32
Freeing unused kernel image (initmem) memory: 1024K
Run /init as init process
  with arguments:
    /init
  with environment:
    HOME=/
    TERM=linux
asgn1_driver: loading out-of-tree module taints kernel.
asgn1_driver: loaded with major 241
asgn1_driver: device opened
asgn1_driver: ASGN1_RESET_BUFFER called
asgn1_driver: ASGN1_GET_VERSION called
asgn1_driver: ASGN1_SET_MODE called
asgn1_driver: stored 9 input bytes in mode 1
asgn1_driver: ASGN1_GET_STATS called
asgn1_driver: ASGN1_SET_MODE called
asgn1_driver: stored 11 input bytes in mode 0
asgn1_driver: ASGN1_RESET_BUFFER called
asgn1_driver: ASGN1_GET_STATS called
asgn1_driver: unsupported ioctl 0x4163
asgn1_driver: device closed
asgn1_driver: unloaded and cleaned up
ASGN1_QEMU_RESULT=PASS
input: ImExPS/2 Generic Explorer Mouse as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@40000000/
bus@40000000:motherboard-bus@40000000:iofpga@7,00000000/10007000.kmi/serio1/input/input2
Flash device refused suspend due to active operation (state 20)
Flash device refused suspend due to active operation (state 20)
reboot: Power down
Task 1 QEMU tests passed. Full output: /home/elaina/embedded_assignment1/logs/test_asgn1_output.txt
```

# Appendix B — Full Task 2 post-test dmesg

```
Task 2 — actual dmesg output after counter_driver cleanup
Booting Linux on physical CPU 0x0
Linux version 5.15.0 (elaina@Lenovo-LOQ-15IAX9) (arm-linux-gnueabihf-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0, GNU
ld (GNU Binutils for Ubuntu) 2.46) #1 SMP Sat Sep 12 15:01:16 +07 2026
CPU: ARMv7 Processor [410fc090] revision 0 (ARMv7), cr=10c5387d
CPU: PIPT / VIPT nonaliasing data cache, VIPT nonaliasing instruction cache
OF: fdt: Machine model: V2P-CA9
Memory policy: Data cache writeback
Reserved memory: created DMA memory pool at 0x4c000000, size 8 MiB
OF: reserved mem: initialized node vram@4c000000, compatible id shared-dma-pool
cma: Reserved 16 MiB at 0x6f000000
Zone ranges:
  Normal   [mem 0x0000000060000000-0x000000006fffffff]
Movable zone start for each node
Early memory node ranges
  node   0: [mem 0x0000000060000000-0x000000006fffffff]
Initmem setup node 0 [mem 0x0000000060000000-0x000000006fffffff]
CPU: All CPU(s) started in SVC mode.
percpu: Embedded 15 pages/cpu s30156 r8192 d23092 u61440
pcpu-alloc: s30156 r8192 d23092 u61440 alloc=15*4096
pcpu-alloc: [0] 0 [0] 1 [0] 2 [0] 3
Built 1 zonelists, mobility grouping on.  Total pages: 65024
Kernel command line: console=ttyAMA0,115200 rdinit=/init
printk: log_buf_len individual max cpu contribution: 4096 bytes
printk: log_buf_len total cpu_extra contributions: 12288 bytes
printk: log_buf_len min size: 16384 bytes
printk: log_buf_len: 32768 bytes
printk: early log buf free: 14964(91%)
Dentry cache hash table entries: 32768 (order: 5, 131072 bytes, linear)
Inode-cache hash table entries: 16384 (order: 4, 65536 bytes, linear)
mem auto-init: stack:all(zero), heap alloc:off, heap free:off
Memory: 229712K/262144K available (8192K kernel code, 593K rwdata, 1784K rodata, 1024K init, 151K bss, 16048K
reserved, 16384K cma-reserved)
SLUB: HWalign=64, Order=0-3, MinObjects=0, CPUs=4, Nodes=1
trace event string verifier disabled
rcu: Hierarchical RCU implementation.
rcu:            RCU event tracing is enabled.
rcu:            RCU restricting CPUs from NR_CPUS=8 to nr_cpu_ids=4.
rcu: RCU calculated value of scheduler-enlistment delay is 10 jiffies.
rcu: Adjusting geometry for rcu_fanout_leaf=16, nr_cpu_ids=4
NR_IRQS: 16, nr_irqs: 16, preallocated irqs: 16
GIC CPU mask not found - kernel will fail to boot.
GIC CPU mask not found - kernel will fail to boot.
L2C: platform modifies aux control register: 0x02020000 -> 0x02420000
L2C: DT/platform modifies aux control register: 0x02020000 -> 0x02420000
L2C-310 enabling early BRESP for Cortex-A9
L2C-310 full line of zeros enabled for Cortex-A9
L2C-310 dynamic clock gating disabled, standby mode disabled
L2C-310 cache controller enabled, 8 ways, 128 kB
L2C-310: CACHE_ID 0x410000c8, AUX_CTRL 0x46420001
random: get_random_bytes called from start_kernel+0x4f8/0x6ac with crng_init=0
sched_clock: 32 bits at 24MHz, resolution 41ns, wraps every 89478484971ns
clocksource: arm,sp804: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 1911260446275 ns
Failed to initialize '/bus@40000000/motherboard-bus@40000000/iofpga@7,00000000/timer@12000': -22
smp_twd: clock not found -2
Console: colour dummy device 80x30
Calibrating local timer... 94.87MHz.
Calibrating delay loop... 1646.59 BogoMIPS (lpj=8232960)
pid_max: default: 32768 minimum: 301
Mount-cache hash table entries: 1024 (order: 0, 4096 bytes, linear)
Mountpoint-cache hash table entries: 1024 (order: 0, 4096 bytes, linear)
CPU: Testing write buffer coherency: ok
CPU0: Spectre v2: using BPIALL workaround
CPU0: thread -1, cpu 0, socket 0, mpidr 80000000
Setting up static identity map for 0x60100000 - 0x60100060
rcu: Hierarchical SRCU implementation.
smp: Bringing up secondary CPUs ...
smp: Brought up 1 node, 1 CPU
SMP: Total of 1 processors activated (1646.59 BogoMIPS).
CPU: All CPU(s) started in SVC mode.
devtmpfs: initialized
VFP support v0.3: implementor 41 architecture 3 part 30 variant 9 rev 0
clocksource: jiffies: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 19112604462750000 ns
futex hash table entries: 1024 (order: 4, 65536 bytes, linear)
NET: Registered PF_NETLINK/PF_ROUTE protocol family
DMA: preallocated 256 KiB pool for atomic coherent allocations
cpuidle: using governor ladder
hw-breakpoint: debug architecture 0x4 unsupported.
Serial: AMBA PL011 UART driver
irq: type mismatch, failed to map hwirq-75 for interrupt-controller@1e001000!
SCSI subsystem initialized
libata version 3.00 loaded.
usbcore: registered new interface driver usbfs
usbcore: registered new interface driver hub
usbcore: registered new device driver usb
pps_core: LinuxPPS API ver. 1 registered
pps_core: Software ver. 5.3.6 - Copyright 2005-2007 Rodolfo Giometti <giometti@linux.it>
PTP clock support registered
Advanced Linux Sound Architecture Driver Initialized.
clocksource: Switched to clocksource arm,sp804
NET: Registered PF_INET protocol family
IP idents hash table entries: 4096 (order: 3, 32768 bytes, linear)
tcp_listen_portaddr_hash hash table entries: 512 (order: 0, 6144 bytes, linear)
TCP established hash table entries: 2048 (order: 1, 8192 bytes, linear)
TCP bind hash table entries: 2048 (order: 2, 16384 bytes, linear)
TCP: Hash tables configured (established 2048 bind 2048)
UDP hash table entries: 256 (order: 1, 8192 bytes, linear)
UDP-Lite hash table entries: 256 (order: 1, 8192 bytes, linear)
NET: Registered PF_UNIX/PF_LOCAL protocol family
RPC: Registered named UNIX socket transport module.
RPC: Registered udp transport module.
RPC: Registered tcp transport module.
RPC: Registered tcp NFSv4.1 backchannel transport module.
Unpacking initramfs...
hw perfevents: enabled with armv7_cortex_a9 PMU driver, 7 counters available
workingset: timestamp_bits=30 max_order=16 bucket_order=0
squashfs: version 4.0 (2009/01/31) Phillip Lougher
jffs2: version 2.2. (NAND) © 2001-2006 Red Hat, Inc.
9p: Installing v9fs 9p2000 file system support
io scheduler mq-deadline registered
io scheduler kyber registered
sii902x 0-0060: supply iovcc not found, using dummy regulator
sii902x 0-0060: supply cvcc12 not found, using dummy regulator
Freeing initrd memory: 1264K
physmap-flash 40000000.flash: physmap platform flash device: [mem 0x40000000-0x43ffffff]
40000000.flash: Found 2 x16 devices at 0x0 in 32-bit bank. Manufacturer ID 0x000000 Chip ID 0x000000
Intel/Sharp Extended Query Table at 0x0031
Using buffer write method
erase region 0: offset=0x0,size=0x40000,blocks=256
physmap-flash 40000000.flash: physmap platform flash device: [mem 0x44000000-0x47ffffff]
40000000.flash: Found 2 x16 devices at 0x0 in 32-bit bank. Manufacturer ID 0x000000 Chip ID 0x000000
Intel/Sharp Extended Query Table at 0x0031
Using buffer write method
erase region 0: offset=0x0,size=0x40000,blocks=256
Concatenating MTD devices:
(0): "40000000.flash"
(1): "40000000.flash"
into device "40000000.flash"
physmap-flash 48000000.psram: physmap platform flash device: [mem 0x48000000-0x49ffffff]
libphy: Fixed MDIO Bus: probed
libphy: smsc911x-mdio: probed
smsc911x 4e000000.ethernet eth0: MAC Address: 52:54:00:12:34:56
isp1760 4f000000.usb: isp1760 bus width: 32, oc: digital
isp1760 4f000000.usb: NXP ISP1760 USB Host Controller
isp1760 4f000000.usb: new USB bus registered, assigned bus number 1
isp1760 4f000000.usb: Scratch test failed. 0x00000000
isp1760 4f000000.usb: can't setup: -19
isp1760 4f000000.usb: USB bus 1 deregistered
usbcore: registered new interface driver usb-storage
ledtrig-cpu: registered to indicate activity on CPUs
usbcore: registered new interface driver usbhid
usbhid: USB HID core driver
NET: Registered PF_PACKET protocol family
9pnet: Installing 9P2000 support
Registering SWP/SWPB emulation handler
aaci-pl041 10004000.aaci: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 32
aaci-pl041 10004000.aaci: FIFO 512 entries
mmci-pl18x 10005000.mmci: Got CD GPIO
mmci-pl18x 10005000.mmci: Got WP GPIO
mmci-pl18x 10005000.mmci: mmc0: PL181 manf 41 rev0 at 0x10005000 irq 33,34 (pio)
10009000.uart: ttyAMA0 at MMIO 0x10009000 (irq = 37, base_baud = 0) is a PL011 rev1
printk: console [ttyAMA0] enabled
1000a000.uart: ttyAMA1 at MMIO 0x1000a000 (irq = 38, base_baud = 0) is a PL011 rev1
1000b000.uart: ttyAMA2 at MMIO 0x1000b000 (irq = 39, base_baud = 0) is a PL011 rev1
```

```
1000c000.uart: ttyAMA3 at MMIO 0x1000c000 (irq = 40, base_baud = 0) is a PL011 rev1
rtc-pl031 10017000.rtc: registered as rtc0
rtc-pl031 10017000.rtc: setting system clock to 2026-11-06T06:13:52 UTC (1793945632)
amba 10020000.clcd: Fixing up cyclic dependency with 0-0039
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
sii902x 0-0039: supply iovcc not found, using dummy regulator
sii902x 0-0039: supply cvcc12 not found, using dummy regulator
i2c i2c-0: Added multiplexed i2c bus 2
drm-clcd-pl111 1001f000.clcd: assigned reserved memory node vram@4c000000
drm-clcd-pl111 1001f000.clcd: using device-specific reserved memory
drm-clcd-pl111 1001f000.clcd: core tile graphics present
drm-clcd-pl111 1001f000.clcd: this device will be deactivated
drm-clcd-pl111 1001f000.clcd: Versatile Express init failed - -19
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
drm-clcd-pl111 10020000.clcd: found bridge on endpoint 0
drm-clcd-pl111 10020000.clcd: Using non-panel bridge
[drm] Initialized pl111 1.0.0 20170317 for 10020000.clcd on minor 0
input: AT Raw Set 2 keyboard as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@40000000/
bus@40000000:motherboard-bus@40000000:iofpga@7,00000000/10006000.kmi/serio0/input/input0
Console: switching to colour frame buffer device 128x48
drm-clcd-pl111 10020000.clcd: [drm] fb0: pl111 frame buffer device
ALSA device list:
  #0: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 32
Freeing unused kernel image (initmem) memory: 1024K
Run /init as init process
  with arguments:
    /init
  with environment:
    HOME=/
    TERM=linux
counter_driver: loading out-of-tree module taints kernel.
counter_driver: registered /dev/counter (major 10, minor 125)
counter_driver: unregistered /dev/counter
input: ImExPS/2 Generic Explorer Mouse as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@40000000/
bus@40000000:motherboard-bus@40000000:iofpga@7,00000000/10007000.kmi/serio1/input/input2
Flash device refused suspend due to active operation (state 20)
Flash device refused suspend due to active operation (state 20)
reboot: Power down
```